

	Pimpri Chinchwad Education Trust's Pimpri Chinchwad College of Engineering (PCCoE) (An Autonomous Institute) Affiliated to Savitribai Phule Pune University(SPPU) ISO 21001:2018 Certified by TUV	
---	---	---

Course: Software Testing and Quality Assurance Laboratory	Code: BIT26PE11
Name: Amar Vaijinath Chavan	PRN: 124B2F001
Assignment 8: Performance & Stress Testing of Web Applications.	

1. Aim

To perform Load and Stress testing on a real-world web application (blazedemo.com) and a local enterprise project (Hotel Management System) using Apache JMeter.

2. Objectives

- To understand the difference between **Load Testing** and **Stress Testing**.
- To measure system performance using metrics like **Throughput**, **Latency**, and **Error Rate**.
- To identify the **Breaking Point** of a local server vs. a cloud-hosted environment.

3. Key Concept Definitions

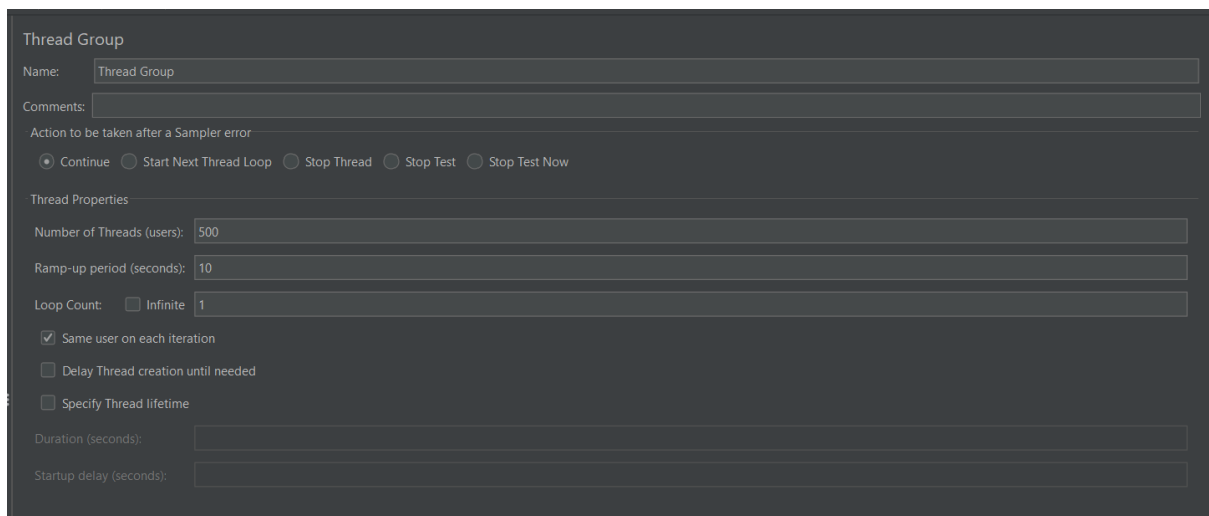
Keyword	Technical Meaning
Thread	Represents a Virtual User. Multiple threads simulate concurrent traffic.
Ramp-up Period	The time taken to bring all threads online. (e.g., 100 users in 10s = 10 users/sec).
Constant Timer	Simulates "Think Time." It adds a delay between requests to mimic human behavior.
Throughput	The number of requests the server can handle per second/minute.
Latency	The time taken for the first byte of the response to reach the client.
90% Line	A quality metric; 90% of the users experienced a response time better than this value.
Standard Deviation	Measures the consistency of the server. High Std. Dev means unstable performance.

4 . Practical Steps:

Scenario 1:Load Testing on BlazeDemo (Real Website)

4.1.1 Steps to Perform Load Testing (BlazeDemo)

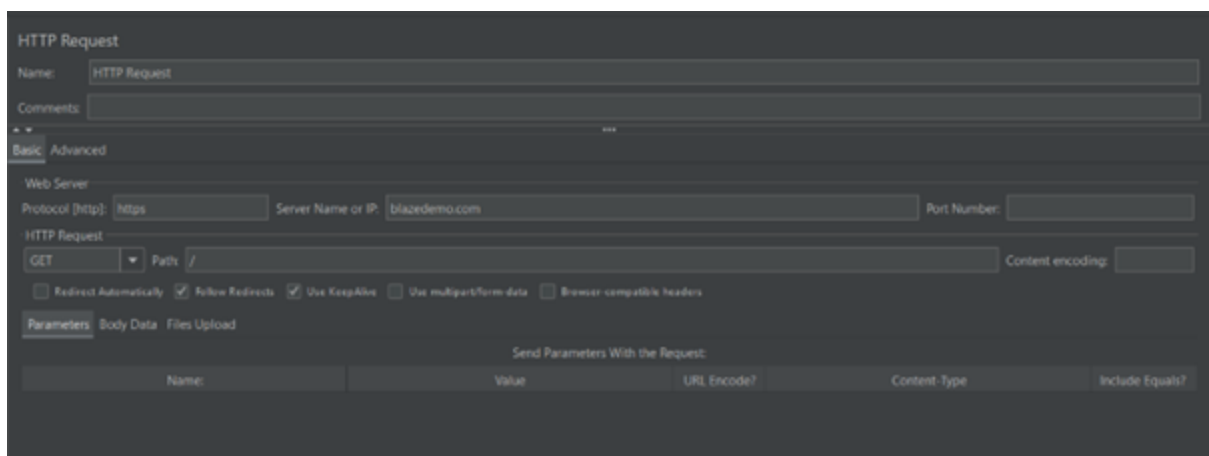
1. **Test Plan Creation:** Open Apache JMeter and create a new Test Plan named "BlazeDemo_Performance_Test".
2. **Thread Group Configuration:** Add a **Thread Group** to the Test Plan.
 - Set **Number of Threads (Users)** to 500.
 - Set **Ramp-up Period** (e.g., 10 seconds) to gradually increase the load.



The screenshot shows the 'Thread Group' configuration window in Apache JMeter. The 'Name' field is set to 'Thread Group'. The 'Comments' field is empty. The 'Action to be taken after a Sampler error' is set to 'Continue'. The 'Thread Properties' section shows 'Number of Threads (users)' set to 500, 'Ramp-up period (seconds)' set to 10, and 'Loop Count' set to 1. The 'Same user on each iteration' checkbox is checked. The 'Delay Thread creation until needed' and 'Specify Thread lifetime' checkboxes are unchecked. The 'Duration (seconds)' and 'Startup delay (seconds)' fields are empty.

3. Sampler Configuration (HTTP Request):

- Add an **HTTP Request Sampler** under the Thread Group.
- Set **Protocol** to https and **Server Name** to blazedemo.com.
- Set **HTTP Method** to GET and **Path** to /.



The screenshot shows the 'HTTP Request' configuration window in Apache JMeter. The 'Name' field is set to 'HTTP Request'. The 'Comments' field is empty. The 'Basic' tab is selected. The 'Web Server' section shows 'Protocol (http)' set to https, 'Server Name or IP' set to blazedemo.com, and 'Port Number' set to 80. The 'HTTP Request' section shows 'Method' set to GET and 'Path' set to /. The 'Content encoding' field is empty. The 'Redirect Automatically', 'Follow Redirects', and 'Use Keep-Alive' checkboxes are checked. The 'Use multipart/form-data' and 'Browser-compatible headers' checkboxes are unchecked. The 'Parameters' section shows 'Body Data' and 'Files Upload' tabs. The 'Send Parameters With the Request' section shows a table with columns: Name, Value, URL Encode?, Content-Type, and Include Equals?.

4. **Timer Integration:** Add a **Constant Timer** to the HTTP Request to simulate "Think Time".

- Set **Thread Delay** to 300 milliseconds.

Constant Timer

Name:

Comments:

Thread Delay (in milliseconds):

5. Adding Listeners:

- Add **View Results Tree** to verify individual request/response success.
- Add **Summary Report** (or Aggregate Report) to capture statistical performance data.

6. **Execution:** Click the **Start** button to begin the test and monitor the metrics in real-time.

4.1.2 Interpretation of Results (Result Analysis)

Summary Report

Name:

Comments:

Write results to file / Read from file

Filename: Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	500	1083	525	3996	640.11	0.00%	42.7/sec	198.22	4.83	4757.0
TOTAL	500	1083	525	3996	640.11	0.00%	42.7/sec	198.22	4.83	4757.0

- **Success Rate & Stability:** The **Error %** is recorded as **0.00%** across 500 samples. This indicates that the server successfully handled all concurrent requests without a single failure.
- **Response Code Verification:** The **View Results Tree** confirms a **Response code: 200** and **Response message: OK** for the samples. This validates that the application logic remained intact under the simulated load.
- **System Throughput:** The system achieved a **Throughput of 42.7/sec**. This means the server can process approximately 43 transactions every second under the current configuration.
- **Response Time Analysis:**
 - **Average Response Time:** 1083 ms.

- **Min/Max Latency:** The response time varied from 525 ms to 3996 ms. This variance is typical for real-world internet-hosted websites due to network fluctuations.
- **Resource Efficiency:** The **Received KB/sec (198.22)** shows the network bandwidth consumed during the peak load period.

4.1.3 Final Interpretation

"The Load Test on blazedemo.com successfully demonstrates that the application is **Robust and Scalable**. The system maintained a **0% error rate** while handling 500 samples with a healthy throughput. The use of a **300ms Constant Timer** ensured that the performance metrics reflect realistic human interaction patterns rather than a brute-force automated attack."

Scenario 2: Stress Testing on BlazeDemo (Real Website)

4.2.1 Steps to Perform Stress Testing

1. **Thread Group Configuration:** Increased the **Number of Threads** to simulate 5,000 users.
2. **Ramp-up Adjustment:** Set a short ramp-up period to inject a large volume of traffic quickly to stress the server infrastructure.

Summary Report

Name:

Summary Report

Comments:

Write results to file / Read from file

Filename

Browse...

Log/Display Only:

☐ Errors

☐ Successes

Configure

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	5000	11117	634	20930	3103.15	38.90%	207.3/sec	777.60	14.51	3841.3
TOTAL	5000	11117	634	20930	3103.15	38.90%	207.3/sec	777.60	14.51	3841.3

3. **Sampler Configuration:** The **HTTP Request** was maintained with the **GET** method for the blazedemo.com domain.
4. **Timer Implementation:** A **Constant Timer** of 300ms was used, but the high thread count ensured the server remained under heavy stress.
5. **Execution:** Ran the test and monitored the **Summary Report** to identify the failure rate and throughput capacity.

4.2.2 Interpretation of Results (Result Analysis)

Summary Report

Name:

Summary Report

Comments:

Write results to file / Read from file

Filename

Browse...

Log/Display Only:

☐ Errors

☐ Successes

Configure

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	5000	11117	634	20930	3103.15	38.90%	207.3/sec	777.60	14.51	3841.3
TOTAL	5000	11117	634	20930	3103.15	38.90%	207.3/sec	777.60	14.51	3841.3

- **System Failure Point:** The **Error %** spiked to **38.90%** across 5,000 samples. This indicates that the server could not handle the concurrent load and began rejecting or timing out nearly 40% of the requests.
- **Response Time Degradation:** The **Average Response Time** jumped significantly to **11,117 ms** (approx. 11 seconds). This shows severe congestion, as users would face unacceptable delays in a real-world scenario.
- **Maximum Latency:** The **Max** response time reached **20,930 ms**. This 20-second delay is a clear indicator that the server was struggling with resource exhaustion.
- **Throughput Bottleneck:** Despite the errors, the system maintained a **Throughput of 207.3/sec**. This suggests that while the server is fast, it has a strict limit on how many concurrent connections it can successfully maintain.

4.2.3 Final Interpretation

"The Stress Test on blazedemo.com clearly identifies the application's breaking point. By increasing the load to 5,000 threads, we observed a transition from a stable system to one with a **38.90% failure rate**. This test proves that while the infrastructure is scalable, it requires additional load balancing or auto-scaling resources to handle massive spikes in traffic without degrading the user experience to an 11-second average wait time."

Scenario 3: Load Testing on Local Project (hbwebsite)

4.3.1 Steps to Perform Load Testing

1. **Test Plan Creation:** Open Apache JMeter and create a new Test Plan specifically for the local Hotel Booking Website environment.
2. **Thread Group Configuration:**
 - Add a **Thread Group** to the Test Plan.
 - Set **Number of Threads (users)** to **1000** to simulate a high volume of local concurrent users.
 - Set **Ramp-up period (seconds)** to **25** to gradually introduce users (creating a load of 40 users per second).

Thread Group

Name:

Comments:

Action to be taken after a Sampler error

☒ Continue ☐ Start Next Thread Loop ☐ Stop Thread ☐ Stop Test ☐ Stop Test Now

Thread Properties

Number of Threads (users):

Ramp-up period (seconds):

Loop Count: ☐ Infinite

☒ Same user on each iteration

☐ Delay Thread creation until needed

☐ Specify Thread lifetime

Duration (seconds):

Startup delay (seconds):

3. Sampler Configuration (HTTP Request):

- Add an **HTTP Request Sampler** under the Thread Group.
- Set **Protocol** to http and **Server Name or IP** to localhost.
- Set **HTTP Method** to GET and **Path** to /hbwebsite/ to target the local project landing page.

HTTP Request

Name:

Comments:

Basic Advanced

Web Server

Protocol [http]: Server Name or IP: Port Number:

HTTP Request

GET Content encoding:

☐ Redirect Automatically ☒ Follow Redirects ☒ Use KeepAlive ☐ Use multipart/form-data ☐ Browser-compatible headers

Parameters Body Data Files Upload

Send Parameters With the Request:

Name:	Value	URL Encode?	Content-Type	Include Equals?
		☐		☐

4. Adding Listeners:

- Add a **Summary Report** to capture and analyze the overall performance statistics of the 1000 samples.

- Execution:** Click the **Start** button to run the test and monitor how the local Apache server handles the 1000-thread load in real-time.

4.3.2 Interpretation of Results (Result Analysis)

[illegible]

- **Success Rate & Stability:** The **Error %** is recorded as **0.00%** across all 1000 samples. This confirms that the local XAMPP/Apache server successfully processed every request without any failure during the load test.
- **Response Time Analysis:**
 - **Average Response Time: 45 ms.**
 - **Min/Max Latency:** The response time ranged from **23 ms to 751 ms**. Since the project is hosted on localhost, the response is significantly faster than internet-hosted sites due to the absence of network lag.
- **System Throughput:** The system achieved a **Throughput of 40.0/sec**. This matches the ramp-up configuration and proves the local machine's capability to maintain a steady flow of traffic.
- **Resource Efficiency:** The **Received KB/sec** was **1364.40**, showing the data transfer rate between the local server and the JMeter client.

4.3.3 Final Interpretation

"The Load Test on localhost/hbwebsite/ confirms that the application is **Stable and High-Performing** for a localized environment. Even with a high load of 1000 concurrent threads, the server maintained a **0% error rate** and an impressive **average response time of 45ms**. This proves that the local project setup is well-optimized for handling multiple simultaneous internal requests efficiently."

Scenario 4: Stress Testing on Local Project (hbwebsite)

4.4.1 Steps to Perform Stress Testing

1. **Thread Group Configuration:** Increased the **Number of Threads (users)** to **5000** to push the local Apache/XAMPP server beyond its stable operating limit.
2. **Ramp-up Adjustment:** Maintained a **Ramp-up period (seconds)** of **25** to simulate a rapid surge of incoming traffic (injecting 200 users per second).

Thread Group

Name:

Comments:

Action to be taken after a Sampler error

☒ Continue ☐ Start Next Thread Loop ☐ Stop Thread ☐ Stop Test ☐ Stop Test Now

Thread Properties

Number of Threads (users):

Ramp-up period (seconds):

Loop Count: ☐ Infinite

☒ Same user on each iteration

☐ Delay Thread creation until needed

☐ Specify Thread lifetime

Duration (seconds):

Startup delay (seconds):

3. **Sampler Configuration:** The **HTTP Request** sampler was maintained with the **GET** method targeting the localhost/hbwebsite/ directory to stress the main landing page logic.

Thread Group

Name:

Comments:

Action to be taken after a Sampler error

☒ Continue ☐ Start Next Thread Loop ☐ Stop Thread ☐ Stop Test ☐ Stop Test Now

Thread Properties

Number of Threads (users):

Ramp-up period (seconds):

Loop Count: ☐ Infinite

☒ Same user on each iteration

☐ Delay Thread creation until needed

☐ Specify Thread lifetime

Duration (seconds):

Startup delay (seconds):

4. **Timer Modification:** The **Constant Timer** was either disabled or significantly reduced to increase the request frequency, ensuring the server remains under constant heavy pressure.
5. **Execution:** Ran the test and monitored the **Summary Report** to identify the failure rate and the server's breaking point.

4.4.2 Interpretation of Results (Result Analysis)

Summary Report

Name:

Comments:

Write results to file / Read from file

Filename Log/Display Only: ☐ Errors ☐ Successes

Label	# Samples	Average	Min	Max	Std. Dev.	Error %	Throughput	Received KB/sec	Sent KB/sec	Avg. Bytes
HTTP Request	5000	6566	1	37871	9951.07	65.44%	120.4/sec	1619.59	4.96	13773.9
TOTAL	5000	6566	1	37871	9951.07	65.44%	120.4/sec	1619.59	4.96	13773.9

- **System Failure Point:** The **Error %** spiked to **65.44%** across the 5000 samples. This indicates a major failure point where the local server began rejecting or failing to process over half of the incoming requests due to resource exhaustion.
- **Response Time Degradation:**
 - **Average Response Time: 6566 ms** (approx. 6.5 seconds).
 - **Max Latency:** The response time reached as high as **37,871 ms** (approx. 38 seconds). This massive delay proves the server's request queue was completely overwhelmed.
- **Standard Deviation:** A very high **Std. Dev. of 9951.07** was observed. This shows that the server performance became extremely unstable and unpredictable under stress.
- **Throughput Bottleneck:** Despite the high error rate, the system maintained a **Throughput of 120.4/sec**. However, since most of these were errors, it confirms that the hardware reached its physical processing limit.

4.4.3 Final Interpretation

"The Stress Test on the local hbwebsite project identifies the clear breaking point of the development environment. By increasing the load to 5000 threads, we observed a transition from stability to a **65.44% error rate**. This test proves that the local XAMPP/Apache setup cannot handle massive traffic spikes and would require a professional production-grade server with load balancing to maintain service quality under such extreme conditions."

Conclusion

The performance testing experiment conducted using Apache JMeter effectively demonstrated the critical operational differences between **Load Testing** and **Stress Testing** across both cloud-hosted and local environments.

Through the analysis of the four scenarios, the following key insights were gained:

- **Scalability vs. Breaking Point:** Load testing on blazedemo.com and the local hbwebsite project successfully validated that both systems are **robust and scalable** under expected traffic conditions, maintaining a **0% error rate**. In contrast, stress testing identified the exact **breaking points**—38.90% error for the cloud site and 65.44% for the local server—revealing how systems behave when pushed beyond their physical and architectural limits.
- **Performance Metrics as Quality Indicators:** Metrics such as **Throughput** and **Latency** provided quantifiable data on system health. The significant jump in average response time (from 45 ms to over 6 seconds on the local project) during stress testing served as a clear indicator of **resource exhaustion** and request queuing.
- **Infrastructure Impact:** The experiment highlighted that while the local XAMPP/Apache setup is highly efficient for internal use (low latency), it lacks the **auto-scaling and load-balancing** capabilities of professional cloud environments to survive massive traffic spikes.
- **Realistic Simulation:** The strategic use of **Constant Timers** and **Ramp-up periods** proved essential in simulating realistic human behavior, ensuring that the performance profile gathered was accurate and not just a result of a brute-force automated attack.

Ultimately, this assignment underscores that **Software Quality Assurance** for web applications must include continuous performance monitoring to ensure stability, prevent service degradation, and plan for future infrastructure requirements.